

Command Nexus (lite) — District Deployment Guide

Staff + Student management, self-hosted

Command Nexus Lite

2026

- What this is
- What you'll be running
- Prerequisites you need to bring
 - Infrastructure
 - Google Workspace
 - SIS (student information system)
 - Google service account setup (~15 min)
 - 1. Create the Google Cloud project
 - 2. Create the service account
 - 3. Enable Domain-Wide Delegation
 - 4. Grant admin roles
 - Installing on the host (~15 min)
 - Prereq installer (one time, as root)
 - Clone + configure
 - First login (~5 min)
 - Configuration walkthrough (~30 min)
 - Branding
 - Buildings
 - Google
 - SIS ingest
 - HR classification map
 - Attendance reports
 - Guidance queue routing
 - Verification checklist
 - Ongoing operations
 - Daily
 - Weekly
 - Monthly
 - Backups
 - Troubleshooting
 - "Container X is unhealthy"
 - "No absence records showing today"
 - "Google API 403 permission denied"
 - "Staff profile shows wrong photo / wrong data"
 - Getting help

What this is

Command Nexus is a self-hosted web app for K-12 district IT operations. This "lite" distribution ships the two core modules — **Staff** and **Student Roster** — plus everything they need to work end-to-end: onboarding workflow, ID cards, guidance queue, attendance reporting, custom-section sync, provisioning, and audit logging.

Everything district-specific in the original full-featured deployment (Paxton door access, HP ProCurve ACL editing, Grandstream phone system, HALO vape sensors, Wave VMS cameras) has been stripped out. The stack here assumes a straightforward Google Workspace + PowerSchool/MetaSolutions environment and configures everything else through the Settings page.

What you'll be running

The stack is 5 Docker containers plus an optional 6th watchdog:

Container	Image	Purpose
nginx	nginx:1.27-alpine	Reverse proxy, TLS termination
postgres	postgres:16	Primary datastore
redis	redis:7-alpine	Session store + job queue
api	built from app/Dockerfile	FastAPI web app
worker	same image as api	ARQ background jobs
scheduler	same image as api	Cron-style job enqueueing
autoheal	willfarrell/autoheal:1.2.0	Restart unhealthy containers automatically

Total working set: ~1.5 GB RAM, ~10 GB disk after 6 months of audit logs. Any Linux VM with 4 GB RAM + 40 GB disk + Docker installed can run this comfortably.

Prerequisites you need to bring

Before you clone the repo, gather these — the first-run script will prompt for them.

Infrastructure

- **A Linux VM or bare-metal host** running Ubuntu 22.04+ / Debian 12+ / RHEL 9+ or similar. Docker Desktop on Mac/Windows works for development but production wants Linux.
- **Docker Engine 24+** and **Docker Compose v2** installed.
- **A domain name** pointing to the host, e.g. `nexus.mydistrict.org`.
- **A TLS certificate** for that domain — Let's Encrypt via certbot works and the compose stack has an nginx snippet for it. Or route through Cloudflare Tunnel and let Cloudflare handle TLS (zero-cert option).
- **Outbound Internet access** from the host to `googleapis.com`, `google.com`, PyPI, Docker Hub, and your MetaSolutions SFTP/Gmail endpoints.

Google Workspace

- **Super-admin access** to your Google Workspace tenant.

- A **project in Google Cloud Console** you can create a service account in.
- Willingness to enable **Domain-Wide Delegation** for that service account against these scopes:

```
https://www.googleapis.com/auth/admin.directory.user
https://www.googleapis.com/auth/admin.directory.user.readonly
https://www.googleapis.com/auth/admin.directory.group
https://www.googleapis.com/auth/admin.directory.group.readonly
https://www.googleapis.com/auth/admin.directory.orgunit
https://www.googleapis.com/auth/admin.directory.orgunit.readonly
https://www.googleapis.com/auth/gmail.readonly
https://www.googleapis.com/auth/gmail.send
https://www.googleapis.com/auth/spreadsheets
https://www.googleapis.com/auth/drive.readonly
```

Nexus impersonates a district admin mailbox for Gmail send/read, and hits Directory API as the service account itself.

SIS (student information system)

Nexus's roster and attendance pipelines are built around **PowerSchool with MetaSolutions ReportingServices** as the export backend. If your district uses the same, no work needed — just point Nexus at the reports.

If you're on a different SIS (Skyward, Aeries, Infinite Campus), the CSV import path is generic — you'll need to map your export's column names onto Nexus's expected schema via Settings. See the SIS section below.

Reports Nexus consumes:

Report	Cadence	Delivery
Students (roster snapshot)	Daily	Emailed CSV to a dedicated mailbox
Daily Attendance	Daily	Emailed CSV
Membership status (A/R/F/CTC/etc.)	Daily	Emailed CSV

Google service account setup (~15 min)

1. Create the Google Cloud project

- Open <https://console.cloud.google.com/>.
- Click the project dropdown at the top, then **New Project**. Name it something like <district>-nexus.
- With the new project active, go to **APIs & Services → Library** and enable:
 - Admin SDK API
 - Gmail API
 - Google Sheets API

- Google Drive API

2. Create the service account

- In the same project, go to **IAM & Admin → Service Accounts → Create Service Account**.
- Name it `nexus-runtime`; you can skip granting Project roles (Nexus doesn't use GCP for compute, only DWD).
- Click into the newly-created account, then **Keys → Add Key → Create new key → JSON**. Save the file — you'll copy it to the Nexus host in a later step as `secrets/google_service_account.json`.

3. Enable Domain-Wide Delegation

- Copy the service account's numeric **Client ID** (starts with 1078... or similar).
- Open <https://admin.google.com>, then go to **Security → Access and data control → API controls → Manage Domain Wide Delegation**.
- Click **Add new**, paste the Client ID, and paste the scopes list from the Prerequisites section above (all on one line, comma-separated).

4. Grant admin roles

Nexus never needs full super-admin. Create a **custom admin role** in `admin.google.com` → Account → Admin roles → Create new role with these privileges, then assign it to the admin mailbox Nexus will impersonate (often a dedicated `nexus.svc@yourdomain.org`):

- Organizational Units (read/write)
- Users (read/write)
- Groups (read/write)
- Reports (read)

Installing on the host (~15 min)

Prereq installer (one time, as root)

If this is a fresh host, the included `scripts/install_prereqs.sh` installs Docker Engine (official repo, not the distro default which is often ancient), the Docker Compose plugin, git, and curl. Works on Ubuntu, Debian, RHEL, Rocky, AlmaLinux, CentOS, and Fedora.

```
# As your regular (non-root) deploy user, in a scratch dir:
curl -fsSL https://raw.githubusercontent.com/timwellington-
alt/command-nexus-lite/master/scripts/install_prereqs.sh \
-o install_prereqs.sh
chmod +x install_prereqs.sh
sudo ./install_prereqs.sh
# Log out + back in so your docker group membership takes effect.
```

The script is idempotent — safe to re-run if it bails partway. Last step adds you to the docker group; you MUST log out + back in before docker commands work without sudo.

Clone + configure

```
# 1. Clone
git clone https://github.com/timwellington-alt/command-
nexus.lite.git \
    command-nexus-lite
cd command-nexus-lite

# 2. Copy the env template
cp .env.example .env
$EDITOR .env      # fill in DISTRICT_NAME, DOMAIN, TIMEZONE

# 3. Drop your Google service account key
cp ~/nexus-runtime-key.json secrets/google_service_account.json
chmod 600 secrets/google_service_account.json

# 4. Run the first-run script – generates DB/redis passwords,
#     session keys, and prompts for the initial admin email.
./scripts/first_run.sh

# 5. Bring up the stack
docker compose up -d

# 6. Watch startup
docker compose logs -f api scheduler worker
```

After ~30 seconds, the stack should be healthy:

```
docker compose ps
```

All containers should read Up (healthy). If any read unhealthy after 2 minutes, tail its logs:

```
docker compose logs --tail 50 <container-name>
```

Common first-boot fixes:

- **Migrations failed → check DB permissions.** The `postgres_password` file must match the actual password Postgres was initialized with; a mismatch after a re-run means you need to `docker compose down -v` to reset the volume.
- **Google API errors → DWD scopes missing.** Re-check the scope list in the Google Admin console.

First login (~5 min)

- Navigate to `https://<your-domain>/`.
- Log in with the admin email you seeded via `first_run.sh`.
- You'll land on `/dashboard`. Most tiles will be dim until you fill out settings — that's expected.

Configuration walkthrough (~30

min)

Go to /settings. Fill these panels in order:

Branding

- **District name** — used in the page header and outbound emails.
- **Logo URL** — small transparent PNG works best (128×128 or 256×256).
- **Primary color** — for accents. Defaults to a neutral blue.

Buildings

- Add each of your schools as a row: Internal code | Display name | SIS code.
- The **Internal code** is what Nexus uses in nav/URLs (short, all-caps, e.g. PES, PHS).
- The **SIS code** is whatever your SIS report emits (e.g. SIS_A, SIS_B) — Nexus keeps a translation table so users never see raw SIS codes.

Google

- **Domain** — your primary Workspace domain (e.g. mydistrict.org).
- **Admin impersonation email** — the mailbox the service account impersonates.
- **Student OU map** — per-building, the OU path new student accounts land in (e.g. PES → /Students/PES).
- **Staff OU map** — same, for staff (e.g. PES → /Staff/PES).

SIS ingest

- **Report sender email** — the address MetaSolutions (or your SIS exporter) sends reports FROM.
- **Roster subject filter** — Gmail search string, e.g. subject:"Students".
- **Attendance subject filter** — e.g. subject:"Daily Attendance".
- **Membership subject filter** — e.g. subject:"Student Membership executed at".

The **Roster ingest** page has a per-column mapping panel where you name each column of your students.csv export against Nexus's expected fields (sis_id, first_name, last_name, grade, school, email, etc.). Once mapped, the next poll picks it up automatically.

HR classification map

- Your HR system uses codes (Cert, Class, Adm, Exempt, etc.) that Nexus maps to role_type (teacher, admin, sub, tech, aide, other).
- Fill this in the HR panel or accept the defaults.

Attendance reports

- **Recipients** — the emails that receive the daily digest, grouped by building.
- **Send time** — hour + minute in district TZ (defaults to 11:00).
- **Weekdays** — Mon-Fri by default.

Guidance queue routing

- **Counselor groups** — building + grade range → counselor email (e.g. PHS grades 9-10 → aparker@...). Nexus uses this to attribute enrollment / withdrawal / transfer queue entries to the right person.

Verification checklist

Before handing off to daily operators, confirm these end-to-end:

- ☐ A user in your admin role logs in and lands on /dashboard.
- ☐ A user in a lower role logs in and DOES NOT see admin-only tiles.
- ☐ /roster renders your students (may be empty on day 1 — poll hasn't run yet).
- ☐ /staff renders your staff directory.
- ☐ Trigger a manual sync from **Settings** → **Google** → **Sync now** and confirm rows populate.
- ☐ Trigger /api/roster/attendance-reports/preview (or the “Preview” button in the Attendance panel) and confirm a sample email renders.
- ☐ docker compose ps shows all containers healthy.
- ☐ /settings **Job Health** panel shows every scheduled job in green after the first tick.

Ongoing operations

Daily

Nothing manual — the scheduler runs everything.

Weekly

- Skim **Settings** → **Job Health**: any red row means a job hasn't succeeded in >3× its cadence. Click into it for the last error.
- Skim **Settings** → **Audit** for anything unexpected.

Monthly

- docker compose pull + docker compose up -d to update to the latest release, then confirm all healthy.
- Rotate the Google service account key if your security policy requires it — drop the new JSON in place, restart api + worker.

Backups

- postgres is the only stateful container. Its data lives in the named volume pg-data.
- Nightly pg_dump recommended; example script in scripts/nightly_backup.sh. Encrypt the dump and ship it off-host.

Troubleshooting

“Container X is unhealthy”

- API: check `docker compose logs api` — usually a DB connection or Google auth error at startup.
- Worker: `docker compose logs worker` — an FD leak or DB drop can wedge it. Autoheal will restart within 60s but the underlying issue is worth reading.

“No absence records showing today”

- Check that today’s Daily Attendance email actually arrived at your Nexus mailbox.
- Manually enqueue: `docker compose exec worker python -c "import asyncio; from app.workers.clever_import_job import poll_clever_imports; print(asyncio.run(poll_clever_imports({})))"`

“Google API 403 permission_denied”

- 99% of the time this is missing or wrong DWD scopes. Recheck the scope list in Google Admin against this doc.

“Staff profile shows wrong photo / wrong data”

- Trigger `run_staff_reconciliation` manually from **Settings → Job Health → Run now**. The reconciler is the arbiter of truth; other syncs feed it.

Getting help

- Issues + questions: open a GitHub issue on the template repo.
- Time-critical outages: this is self-hosted; there is no vendor support line. Roll back with `docker compose down + revert to the last known-good tag with git checkout <tag> + docker compose up -d`.